Market-Basket Analysis of the IMDB Top 1000 Dataset

Distributed Frequent Itemset Mining using PySpark

Moein Rajabi Ghaemiyeh

Algorithms for Massive Data

Instructor: Prof. Malchiodi

June, 2026

Anti-Plagiarism Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1 Dataset and Data Organization

1.1 The Dataset

For this project, the "IMDB Dataset of Top 1000 Movies and TV Shows" dataset is used which is publicly available via Kaggle. This dataset contains some information about 1000 movies, including attributes such as title, genre, overview, and primary cast members.

To perform a Market-Basket Analysis on this dataset, each movie is considered as a basket, and its four cast members as items. The goal is to identify the frequent collaborations among different actors in different movies. For this purpose, the vast majority of dataset's information is discarded, and only the information about cast members is kept (columns named `Star1`, `Star2`, `Star3`, and `Star4`).

1.2 Data Organization and Market-Basket Mapping

To apply the Apriori algorithm, the extracted tabular data was conceptually mapped to the standard Market-Basket Analysis paradigm. In its traditional context, a transaction consists of a basket of items purchased together by a customer. In the case of this project, the data was organized as follows:

- **Transactions (T):** Each individual movie represents a transaction.
- **Items (I):** The individual actors or "stars" represent the items in that transaction.
- **Itemsets (Baskets):** The aggregate of the four `Star` columns for a given movie constitutes the "basket" of items for that movie. As a result, the absolute maximum size of any basket in this specific dataset is bounded at $k = 4$.

By structuring the data in this manner, collaborative occurrences between actors are treated identically to items co-occurring in a shopping cart. This allows for the mathematical extraction of frequent itemsets and subsequent association rules regarding Market-Basket Analysis.

2 Pre-Processing Techniques

Before applying the Apriori algorithm, the dataset required some pre-processing to be suitable for distributed Market-Basket Analysis.

2.1 Dynamic Data Subsampling

To make testing faster on large datasets, a data sampling step was added to the code. By using a $use\_subsample$ flag, the program can take a random, smaller piece of the data (for example, 10% if $subsample\_fraction = 0.1$). This random selection uses a fixed seed to ensure reproducibility. Given the dataset's scale, this flag was set to False for this project.

2.2 Data Cleaning and Standardization

To ensure consistency between actors names and avoid duplicate values for a specific actor due to different spacing or uppercase/lowercase differences, `lower()` and `trim()` functions of PySpark were used. After that, all four columns were aggregated into a single array structure using the `array()` function to create the "basket" for each movie.

Subsequently, for handling null values (if any) within the array, `expr()` function was used. To make sure that each basket has at least one item, the baskets were filtered for size greater than zero. Finally, to prevent duplication within a basket (the same actor being repeated accidentally), `array_distinct()` was applied to each basket.

2.3 Integer Mapping

The handling of strings (names of actors) in MapReduce operations consumes much more memory and bandwidth than integers. For this reason, before implementing the algorithm, a mapping was performed in which each actor was represented by a unique 64-bit integer number. Then, the entire RDD was mapped to these numbers using the broadcast method of PySpark. After that, A reverse-mapping dictionary (`id_to_actor`) was maintained to translate the final association rules back into the actual actors' names.

```
# Map string names to integers to save memory and network bandwidth during
    MapReduce
actor_to_id = dict(
    imdb_rdd.flatMap(lambda x: x)
    .distinct()
    .zipWithIndex()
    .collect()
)

# Broadcast the dictionary for efficient access on worker nodes
actor_to_id_bc = sc.broadcast(actor_to_id)
imdb_rdd_id = imdb_rdd.map(lambda x: [actor_to_id_bc.value[actor] for actor in
    x])
```

Snippet 1: Broadcasting and Mapping Strings to Integer IDs

3 Algorithms and Implementations

3.1 The Apriori Algorithm

The core algorithm selected for this frequent itemset mining task is the Apriori algorithm. Apriori relies on this basis that for any frequent itemset, all of its subsets must also be frequent. The algorithm was executed iteratively from L_1 (frequent single actors) up to L_4 (frequent 4-actor tuples). For each iteration, a set of possible candidates (C_k) was produced based on previous frequent item set. Because the maximum basket size in this dataset is exactly four, the algorithm inherently terminates after the fourth iteration, as no C_5 candidates can exist.

3.2 Distributed Counting via MapReduce

```
def counting(data_rdd, min_support):

  # Counts item occurrences in an RDD.
  # Filters out any items that do not meet the minimum support threshold.

  return (
      data_rdd.flatMap(lambda x: x)
      .map(lambda x: (x, 1))
      .reduceByKey(lambda x, y: x + y)
      .filter(lambda x: x[1] >= min_support)
      )
```

Snippet 2: Distributed MapReduce Support Counting Function

To count the support of itemsets across the Spark cluster, a standard MapReduce algorithm was implemented. For a given candidate set C_k, the worker nodes filter the movie baskets to retain only valid candidates. The `flatMap` transformation flattens the resulting combinations, mapping each valid k-length tuple to a key-value pair of `(itemset, 1)`. Finally, by using the `reduceByKey` function, the counts were aggregated in parallel across the network. Itemset were filtered to meet the predefined `min_support` (s) threshold before the next iteration. Although a typical s would be around 1% of the number of baskets, setting $s = 10$ in this dataset results

in very few frequent itemsets and zero association rules. To have a reasonable frequent itemsets and association rules, s was set to 3.

3.3 Candidate Extraction

```
def k_length_candidates(frequent_k_minus_1, k):

  # Generates k-length candidates using Apriori logic.
  # Combines frequent (k-1)-length items to create candidates and prunes those
    ones that have a subset which is not frequent.

  frequent_k_minus_1 = list(frequent_k_minus_1)
  frequent_set = set(frequent_k_minus_1)
  candidates = []
  for i in range(len(frequent_k_minus_1) - 1):
    item1 = frequent_k_minus_1[i]
    for j in range(i + 1, len(frequent_k_minus_1)):
      item2 = frequent_k_minus_1[j]
      if item1[:k - 2] != item2[:k - 2]: # Join only if the first k-2 elements
    match.
        break
      else:
        new_candidate = item1[:k - 2] + (item1[-1], item2[-1])
        flag = 1
        for z in combinations(new_candidate, k - 1): # Ensure all k-1 subsets
    exist in the frequent set.
          if z not in frequent_set:
            flag = 0
            break
        if flag == 1:
          candidates.append(new_candidate)
  return candidates
```

Snippet 3: Apriori Candidate Pruning using itertools.combinations

To find candidate pairs after finding the frequent items, any item that was not frequent was discarded from the basket. With the remaining items in each basket, every possible pair combination was constructed. Using the same logic of counting with the MapReduce approach, frequent pairs were recognized.

In order to construct the candidate lists for triples and 4-tuples, first, frequent items from the previous step were examined. To construct a candidate of length k, frequent items of length $k-1$, which were similar to each other except for the last element, were joined. Then a filtering phase was conducted using Python's `itertools.combinations` library to check whether every possible combination of length $k-1$ of the candidate was also frequent itself.

3.4 Association Rule Generation and Evaluation

Once the frequent itemsets were identified, the algorithm generated directional association rules in the format of $A \rightarrow B$. During the Apriori run, a comprehensive dictionary of the frequent items of all possible sizes was produced to be used in association rule generation. Again, `itertools.combinations` was used to construct every possible association rule from a single frequent itemset. To evaluate the strength and validity of these rules, two statistical metrics were calculated:

1. **Confidence:** The conditional probability that consequent B appears in a transaction given that antecedent A is present.

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)} \tag{1}$$

In this project, the minimum confidence required for a rule to be valid was considered 0.8.

2. **Interestingness:** While Confidence measures the relative strength of the rule, it does not account for the baseline popularity of the consequent. To prevent globally frequent actors from artificially inflating rule validity, Interestingness was employed. It measures the absolute difference between the conditional probability of B and the overall probability of B in the entire dataset of N transactions.

$$\text{Interestingness}(A \to B) = \left| \frac{\text{Support}(A \cup B)}{\text{Support}(A)} - \frac{\text{Support}(B)}{N} \right| \tag{2}$$

The higher the interestingness, the more valid an association rule is. The rules were filtered for having an interestingness greater than 0.7.

4 Scalability of the Proposed Solution

To ensure the Apriori scales to handle massive datasets, the solution relies on Apache Spark, which distributes the computations and does very well in memory and network optimizations. Also, some other optimizations were implemented:

4.1 Memory Storage Optimization: The Sparsity Threshold

Storing candidate pair counts in main memory is one of the main challenges in the Apriori algorithm. There are two options with respect to this matter, each suitable for a different type of dataset.

1. **The Triples Method:** This method saves both i and j (pair) and the count. So, it needs to store 3 integers for each pair. Its benefit is that no memory is used for pairs with zero count.

2. **The Triangular-Matrix Method:** In this method a 1-D array is constructed in a way that from the index the relative i and j can be calculated (so there is no need to save those numbers) and only the pair's count needs to be saved. So, only one integer per pair is saved in memory. However, if the data is sparse, many unnecessary 0s will also be saved.

If at least 1/3 of the possible pairs are present, the Triangular-Matrix would be better. Otherwise, Triples method is more efficient. To choose the right method, 5% of the dataset was sampled and examined. The result shows that the density is far below 1/3 and the data is sparse, meaning the vast majority of actors in the dataset never played in the same movie. So, the Triple method was used throughout the algorithm.

4.2 Network Minimization via Broadcast Variables

```
1 def frequent_pairs(data_rdd, frequent_items, min_support):
2
3   # Generates candidate pairs and counts length-2 frequent pairs using a manual
      nested loop.
4   # Uses a broadcast variable to efficiently filter out non-frequent items.
5
6   frequent_items_bc = sc.broadcast(set(frequent_items))
7   two_pairs_candidates = data_rdd.map(lambda x: [i for i in x if i in
      frequent_items_bc.value]) # Keep only actors that passed the L1 support
      threshold.
8   frequent_pairs = two_pairs_candidates.map(lambda x: [tuple(sorted([x[i], x[j
      ]])) for i in range(len(x) - 1) for j in range(i + 1, len(x))]) # Manually
      generate all possible unique pairs from the remaining actors.
```

```
9   return counting(frequent_pairs, min_support)
```

Snippet 4: Network-Optimized Candidate Pair Generation

In a distributed computing environment, using variables, such as integer mapping dictionary or candidate lists, in lambda functions requires transmitting them through the network each time. To avoid this network usage, these variables were converted to `sc.broadcast()` variables. This ensures that such variables are transmitted exactly once and cached locally in the memory of each worker node.

5 Minimum Support Tuning and Sensitivity Analysis

The minimum support threshold is a key hyperparameter in Apriori algorithm. Setting it too low results in a lot of noise and computational load, while setting it too high can lead to having zero frequent itemsets.

To determine the optimal threshold, a sensitivity analysis was performed by iteratively executing the entire Apriori for $s \in [2, 10]$. During each iteration, the total number of extracted frequent single items (L_1), frequent pairs (L_2), frequent triples (L_3), frequent 4-tuples (L_4), and the resulting valid association rules were recorded.

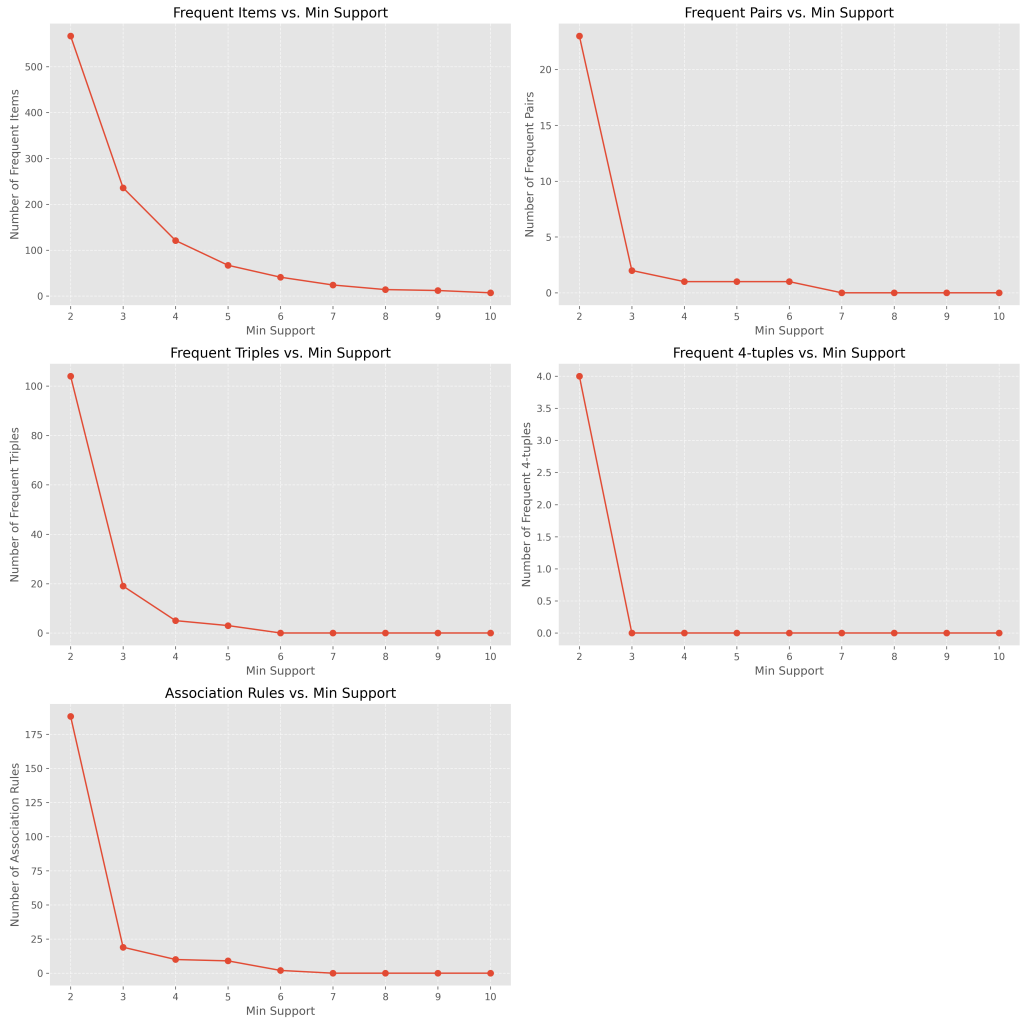

Figure 1: The effect of the Minimum Support threshold on the extraction of frequent itemsets and association rules.

As illustrated in Figure 1, the algorithm's output is highly sensitive to the support threshold.

Because the colaborations in the dataset are highly sparse, raising the support threshold causes a significant drop in itemset discovery.

For example, at $s = 7$, the extraction of rules reaches absolute zero. On the other hand, setting the threshold to the absolute minimum ($s = 2$) results in a spike across all metrics, capturing actor pairings that are statistically weak.

As a result, $s = 3$ was selected as the optimal threshold for the final execution. This value successfully filters out noisy, incidental collaborations while extracting meaningful, multi-actor collaborations.

6 Description of the Experiments

The distributed PySpark application was executed with a minimum support threshold of $s = 3$, meaning an itemset must appear in at least three distinct movies to be considered frequent. The association rule generation was constrained by a minimum Confidence threshold of 0.8 and a minimum Interestingness of 0.7. Also, the *use_subsample* boolean flag was set to False to conduct the analysis on the full dataset. To ensure replicability, the experiment was conducted in a local Spark environment utilizing Google Colab, Apache Spark 3.5.1, and Java 17 to simulate the MapReduce framework.

7 Experimental Results

The execution of the Apriori pipeline yielded the below collaborative networks which are ranked based on the interestingness:

```
Elijah Wood                    ---> Ian Mckellen, Orlando Bloom   | C: 1.00 | Int: 1.00
Orlando Bloom                  ---> Elijah Wood, Ian Mckellen     | C: 1.00 | Int: 1.00
Elijah Wood, Ian Mckellen      ---> Orlando Bloom                 | C: 1.00 | Int: 1.00
Ian Mckellen, Orlando Bloom    ---> Elijah Wood                   | C: 1.00 | Int: 1.00
Mark Hamill                    ---> Harrison Ford, Carrie Fisher  | C: 1.00 | Int: 1.00
Harrison Ford, Carrie Fisher   ---> Mark Hamill                   | C: 1.00 | Int: 1.00
Elijah Wood                    ---> Orlando Bloom                 | C: 1.00 | Int: 1.00
Orlando Bloom                  ---> Elijah Wood                   | C: 1.00 | Int: 1.00
Mark Hamill, Harrison Ford     ---> Carrie Fisher                 | C: 1.00 | Int: 1.00
Mark Hamill                    ---> Carrie Fisher                 | C: 1.00 | Int: 1.00
Daniel Radcliffe, Emma Watson  ---> Rupert Grint                  | C: 1.00 | Int: 0.99
Emma Watson, Rupert Grint      ---> Daniel Radcliffe              | C: 1.00 | Int: 0.99
Daniel Radcliffe               ---> Rupert Grint                  | C: 1.00 | Int: 0.99
Rupert Grint                   ---> Daniel Radcliffe              | C: 1.00 | Int: 0.99
John Turturro                  ---> Ethan Coen                    | C: 1.00 | Int: 0.99
Elijah Wood, Orlando Bloom     ---> Ian Mckellen                  | C: 1.00 | Int: 0.99
Elijah Wood                    ---> Ian Mckellen                  | C: 1.00 | Int: 0.99
Orlando Bloom                  ---> Ian Mckellen                  | C: 1.00 | Int: 0.99
Mark Hamill, Carrie Fisher     ---> Harrison Ford                 | C: 1.00 | Int: 0.99
Mark Hamill                    ---> Harrison Ford                 | C: 1.00 | Int: 0.99
Tim Allen                      ---> Tom Hanks                     | C: 1.00 | Int: 0.99
Daniel Radcliffe               ---> Emma Watson, Rupert Grint     | C: 0.83 | Int: 0.83
Rupert Grint                   ---> Daniel Radcliffe, Emma Watson | C: 0.83 | Int: 0.83
Daniel Radcliffe, Rupert Grint ---> Emma Watson                   | C: 0.83 | Int: 0.83
Daniel Radcliffe               ---> Emma Watson                   | C: 0.83 | Int: 0.83
Rupert Grint                   ---> Emma Watson                   | C: 0.83 | Int: 0.83
```

Association Rules Ranked by Interestingness

Some of these association rules capture the collaborative networks in some of the most famous movies:

1. **The Lord of the Rings:** Elijah Wood, Orlando Bloom, Ian McKellen
2. **Star Wars:** Mark Hamill, Harrison Ford, Carrie Fisher
3. **Harry Potter:** Daniel Radcliffe, Emma Watson, Rupert Grint

The rule `Tim Allen` → `Tom Hanks` captures the collaboration of these two as the lead voice actors in Pixar's *Toy Story* franchise. Also, the rule `John Turturro` → `Ethan Coen` highlights the Coen brothers' tendency to cast Turturro in multiple top-rated films (e.g., *The Big Lebowski*, *O Brother, Where Art Thou?*).

7.1 Confidence and Interestingness Metrics

As observed, the Confidence and Interestingness scores for these rules are nearly identical. Because the dataset is highly sparse, the baseline probability of any specific actor appearing ($P(B)$) is extremely low. Consequently, subtracting this negligible probability from the conditional probability yields an Interestingness score that closely mirrors the Confidence metric.

8 Conclusion

In this project a distributed Market-Basket Analysis pipeline was implemented to discover frequent collaborative actor networks within the IMDB Top 1000 dataset. By leveraging the Apache Spark framework, the Apriori algorithm was transformed into a horizontally scalable MapReduce architecture capable of processing extensive itemsets.

Besides using MapReduce architecture, some techniques such as integer mapping, broadcast variables, and Triples method for sparse matrix storage were employed to minimize memory usage and network I/O. Furthermore, the sensitivity analysis of the minimum support threshold ensured an optimal balance between computational load and analytical depth.

Ultimately, the pipeline yielded some association rules, identifying cinematic clusters such as the core casts of *The Lord of the Rings*, *Star Wars*, and *Harry Potter*. Using both confidence and interestingness measures to filter out association rules resulted in extracting significant collaborations rather than coincidental occurrences.

Overall, this project validates the power of distributed computing paradigms and careful memory management in extracting meaningful, statistically sound insights from highly sparse real-world data.